What Are Transformers Used For?

Applications and Architecture Overview
AIMS Cameroon Deep Learning Course

Prof. Olushina Olawale Awe
AIMS cameroon Deep Learning Course

December 17, 2025

Revision: Why Standard RNNs Struggle

- RNNs process sequences one step at a time.
- They rely solely on the hidden state to pass information forward.
- This leads to:
  - **Vanishing gradients:** long-term dependencies are forgotten.
  - **Exploding gradients:** unstable training.
- As a result, RNNs often fail to capture patterns that span many time steps.

The Rise of LSTMs

Long Short-Term Memory networks (LSTMs) were introduced to solve the memory limitations of RNNs.

- They contain a **cell state** that acts as long-term memory.
- They use **gates** (forget, input, output) to control:
  - What to remember
  - What to forget
  - What to output
- LSTMs can capture long-range dependencies effectively.

Analogy

LSTMs behave like **skilled librarians**: they know what information to keep on the main shelf and what to discard.

The Rise of GRUs

Gated Recurrent Units (GRUs) simplify the LSTM architecture.

- They merge gates (update gate combines roles of forget and input gates).
- Fewer parameters $\rightarrow$ faster training.
- Still very effective at modeling long-term dependencies.

Analogy

GRUs are like **LSTMs' efficient younger siblings**— they perform similar tasks but with a simpler workflow.

The Sequential Bottleneck

Even with LSTMs and GRUs:

- Sequence is still processed **step-by-step**.
- Hard to parallelize computations.
- Slow training on long sequences.
- Limited use of modern GPU/TPU parallel hardware.

Key Limitation

Sequential models cannot use full parallel processing. This inspired the search for a new architecture.

Transformers Enter: A Breakthrough

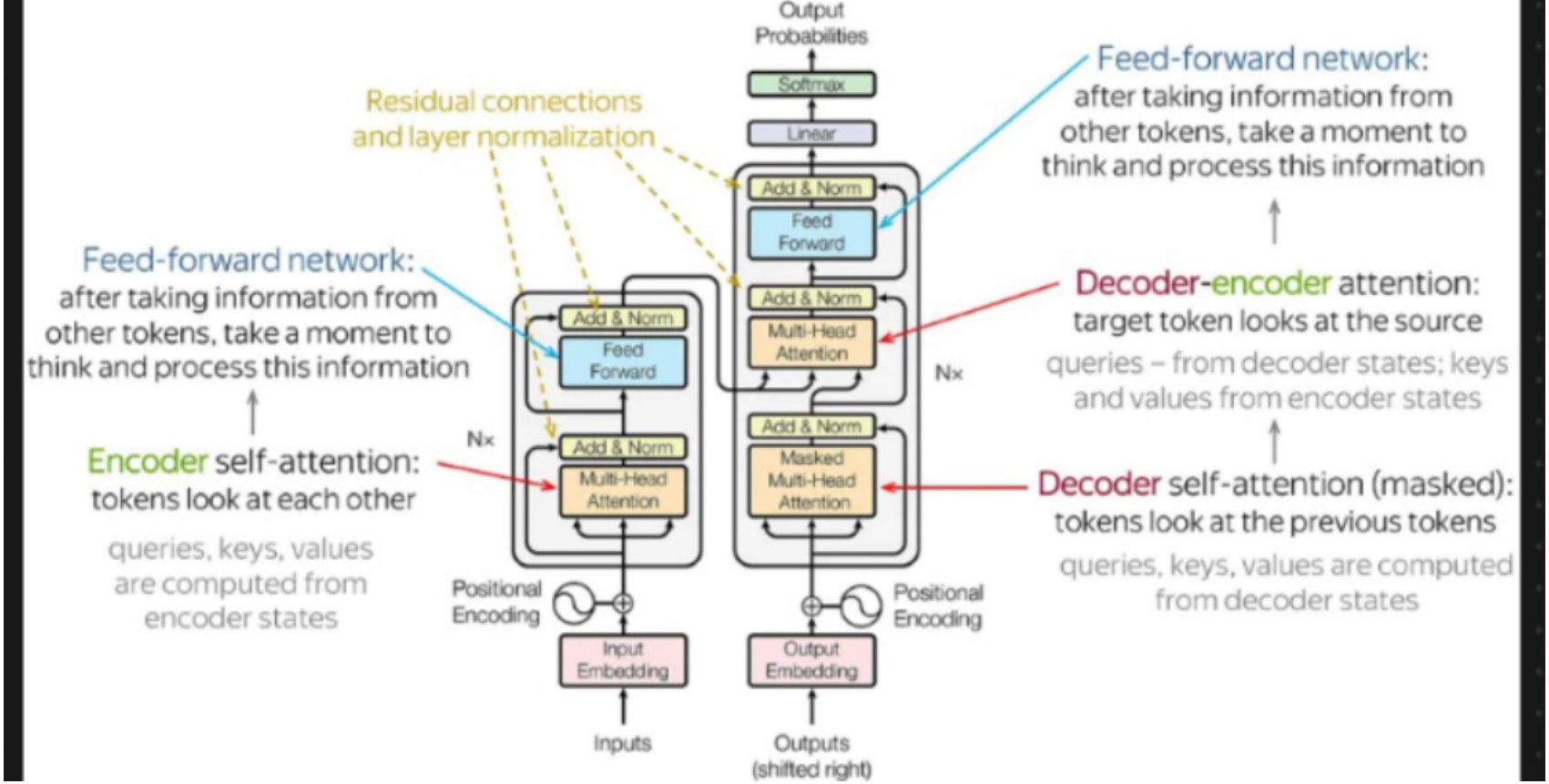
Transformers were introduced in the paper **"Attention is All You Need" (2017)**.

- They eliminate recurrence entirely.
- Use **self-attention** to capture relationships across the entire sequence.
- Entire sequences can be processed **in parallel**.
- Better at modeling long-range dependencies than LSTMs/GRUs.

Impact

Transformers enable:

- Faster training
- Better scalability
- Superior performance on NLP and many other tasks


Output Probabilities
Softmax
Linear
Residual connections and layer normalization
Feed-forward network:
after taking information from other tokens, take a moment to think and process this information
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Add & Norm
Masked Multi-Head Attention
Nx
Decoder-encoder attention:
target token looks at the source
queries – from decoder states; keys and values from encoder states
Feed-forward network:
after taking information from other tokens, take a moment to think and process this information
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Nx
Encoder self-attention:
tokens look at each other
queries, keys, values are computed from encoder states
Decoder self-attention (masked):
tokens look at the previous tokens
queries, keys, values are computed from decoder states
Positional Encoding
Positional Encoding
Input Embedding
Output Embedding
Inputs
Outputs (shifted right)

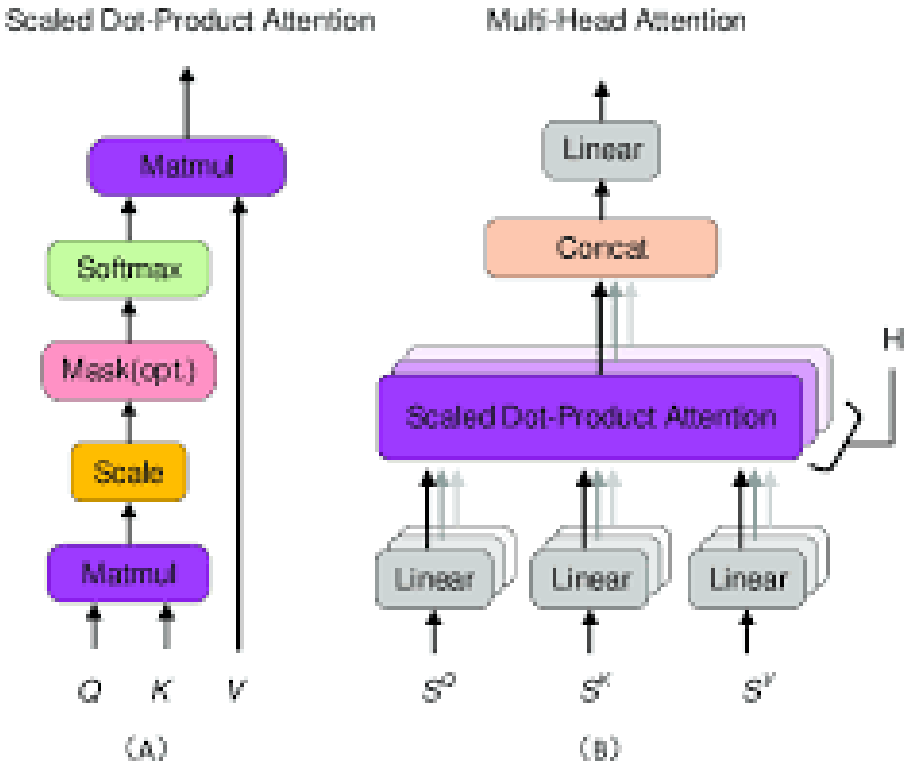


Figure: Simpler Transformer Architecture

Key Advantages of Transformers

- **Self-Attention:** Learns global interactions across sequences
- **Parallelization:** Much faster than RNNs/LSTMs
- **Scalability:** Supports extremely large models (GPT-4, PaLM)
- **Modality Flexibility:** Works on text, images, audio, time-series
- **Long-Range Dependency Learning:** No limitation of sequence length

Self-Attention: Core Equations

Each input token x_i is transformed into:

$$Q_i = x_i W^Q, \quad K_i = x_i W^K, \quad V_i = x_i W^V$$

Attention weights:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

Multi-head attention:

$$\text{MHA}(X) = \bigoplus_{h=1}^{H} \text{Attention}(Q_h, K_h, V_h) W^O$$

Transformer encoder layer:

$$Z_1 = \text{LayerNorm}(X + \text{MHA}(X))$$
$$Z_2 = \text{LayerNorm}(Z_1 + \text{FFN}(Z_1))$$

Text to Tokens

Raw sentence:

"Deep learning is powerful"

Tokenization:

["Deep", "learning", "is", "power", "ful"]

Each token will be converted into a vector.

Turning Words into Vectors

1. Text is tokenized
2. Tokens are mapped to IDs
3. IDs index an embedding matrix
4. Vectors represent meaning
5. Contextual embeddings evolve via attention
6. Word embeddings and self-attention play complementary but different roles in transformer models.
7. Self-attention takes embeddings as input and re-weights them based on context.

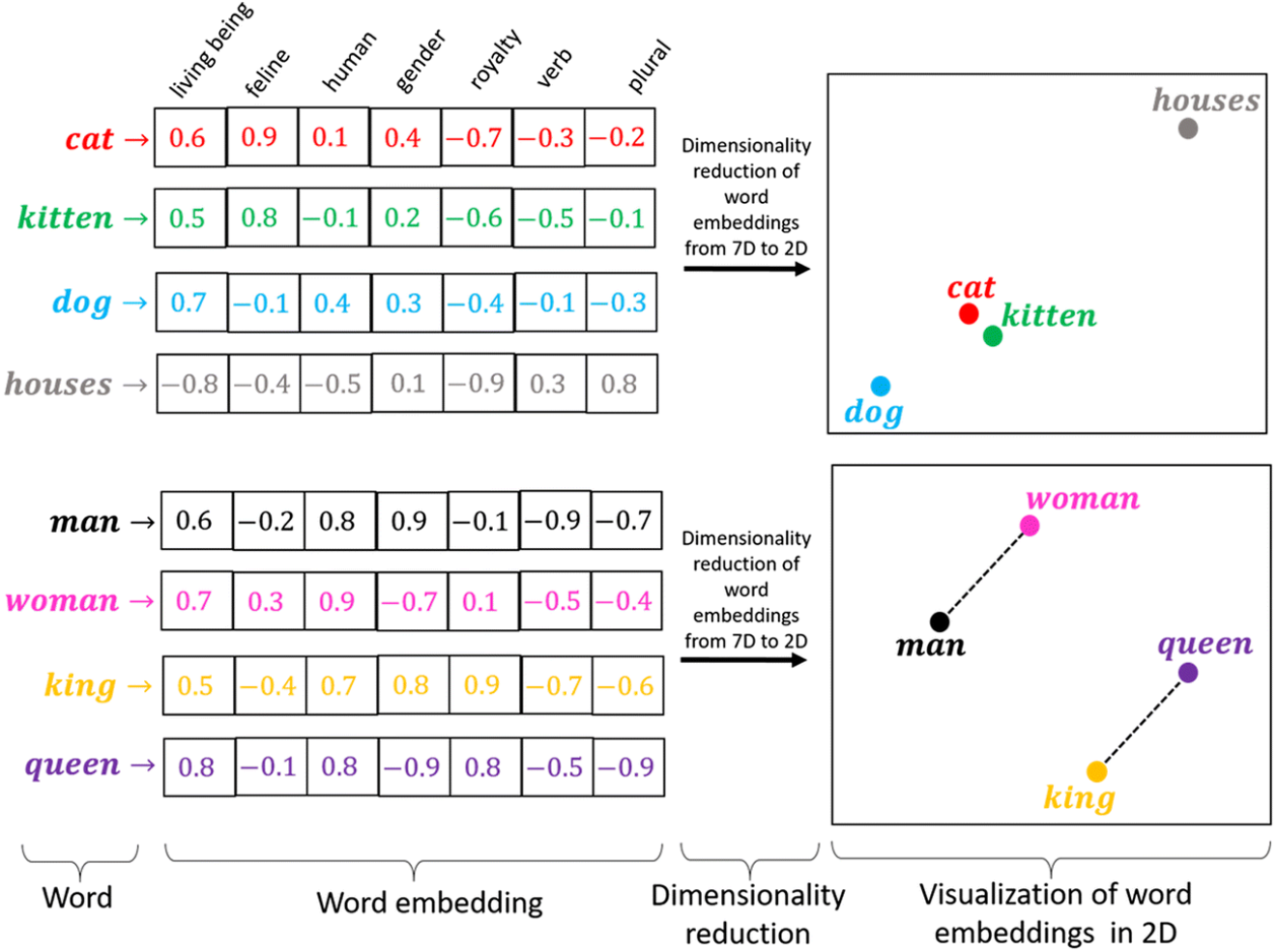
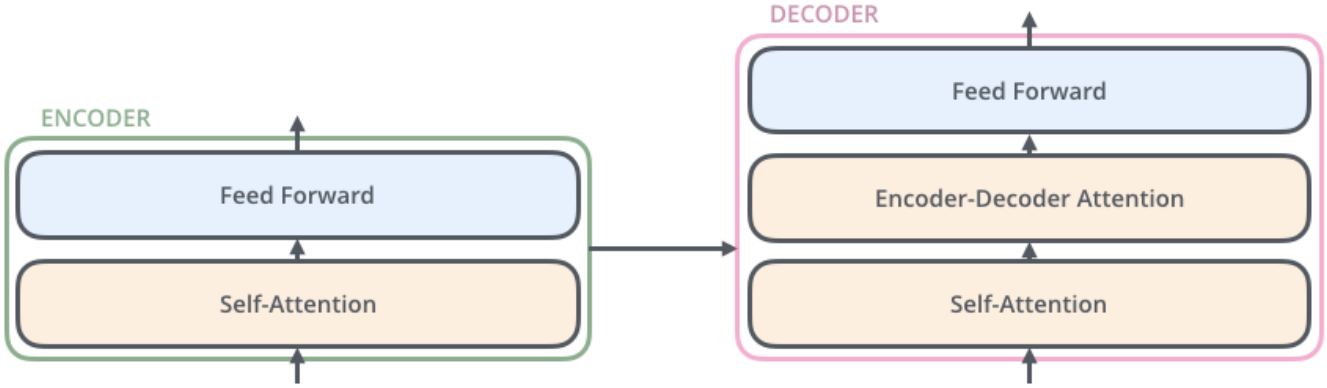


Figure: Word Embedding Example


DECODER
Feed Forward
Encoder-Decoder Attention
Self-Attention
ENCODER
Feed Forward
Self-Attention

What is Attention?

Attention allows a model to focus on the most relevant parts of a sequence when producing an output.

- Computes a weighted combination of all input tokens.
- Higher weights = more important information.
- Allows long-range dependencies without recurrence.

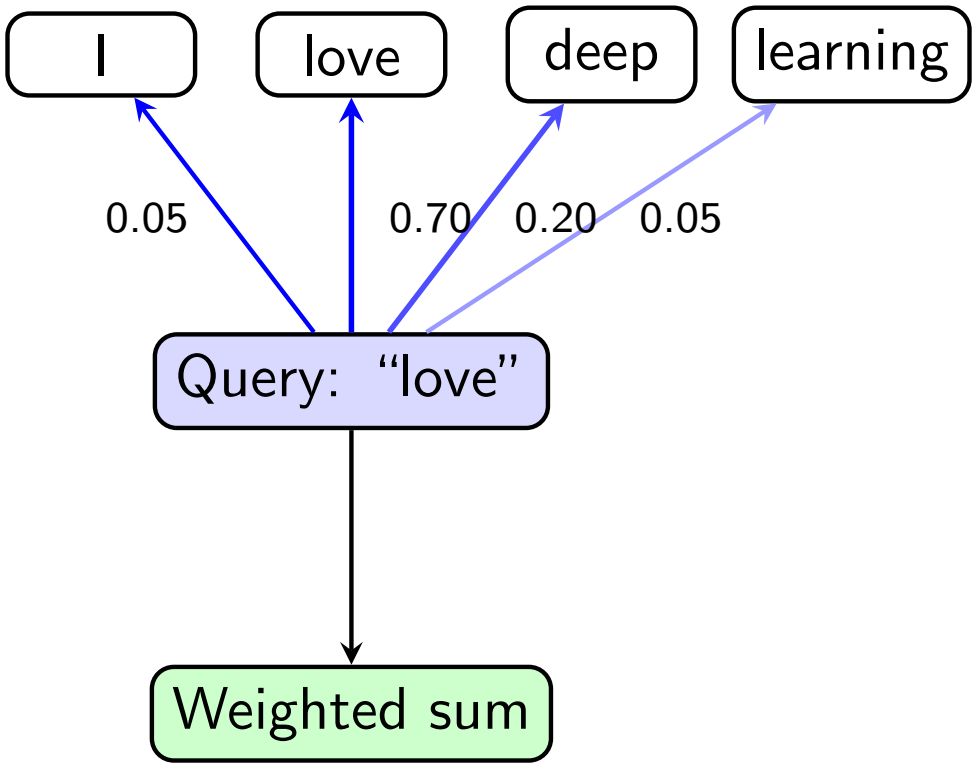
Mathematically:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

Where:

- Q = Query vectors
- K = Key vectors
- V = Value vectors
- $\sqrt{d_k}$ = scaling factor

Attention Visualization



Interpretation: The word *"love"* pays the most attention to itself (70%), some to *"deep"* (20%), and very little to other words. This produces a context-aware representation.

Self-Attention Recap: Examples

Given input representations $X \in \mathbb{R}^{n \times d}$ (n tokens, d-dim):

$$Q = XW^Q, \qquad K = XW^K, \qquad V = XW^V,$$

where $W^Q, W^K, W^V \in \mathbb{R}^{d \times d_k}$.
Score matrix:

$$S = \frac{QK^\top}{\sqrt{d_k}} \in \mathbb{R}^{n \times n}.$$

Row-wise softmax attention weights:

$$A = \text{softmax}(S) \quad \text{(row-wise)}.$$

Output of attention:

$$O = AV \in \mathbb{R}^{n \times d_k}.$$

Softmax Derivative (Per Row)

Consider one row of scores $s \in \mathbb{R}^n$:

$$a_i = \text{softmax}(s)_i = \frac{e^{s_i}}{\sum_j e^{s_j}}.$$

Jacobian of softmax:

$$\frac{\partial a_i}{\partial s_j} = a_i(\delta_{ij} - a_j).$$

In vector form:

$$\frac{\partial a}{\partial s} = \text{diag}(a) - aa^\top.$$

This term will appear when we compute

$$\frac{\partial \mathcal{L}}{\partial S}$$

for the attention layer.

Transformer Block: Big Picture

Each Transformer layer consists of two core components:

1. Multi-Head Self-Attention
2. Position-wise Feed-Forward Neural Network (FFNN)

Architecture Pattern:

$$\text{Attention} \rightarrow \text{Add \& Norm} \rightarrow \text{FFNN} \rightarrow \text{Add \& Norm}$$

Key Message

Core Intuition

Attention mixes information across tokens.
FFNN transforms information within each token.

Both are essential for a Transformer to work.

What is the FFNN in a Transformer?

The Transformer FFNN is:

- A small multilayer perceptron (MLP)
- Applied independently to each token
- With shared parameters across all positions

Position-wise means:

$$h'_i = \text{FFNN}(h_i) \quad \forall i$$

Mathematical Form of Transformer FFNN

For a token representation $x \in \mathbb{R}^{d_{\text{model}}}$:

$$\text{FFNN}(x) = \sigma(xW_1 + b_1)W_2 + b_2$$

Where:

- $W_1 \in \mathbb{R}^{d_{\text{model}} \times d_{ff}}$
- $W_2 \in \mathbb{R}^{d_{ff} \times d_{\text{model}}}$
- $\sigma = \text{ReLU or GELU}$

Why FFNN Is Necessary

Without the FFNN:

- The Transformer becomes nearly linear
- Attention only computes weighted averages
- Model expressiveness collapses

Key Intuition

Attention decides *where to look*.
FFNN decides *what to compute*.

FFNN in Transformers vs MLP Classifier

Aspect	**Transformer FFNN**	**MLP Classifier**
Input	Token representation	Entire feature vector
Scope	Per token	Global input
Parameter sharing	Across positions	No sharing
Context awareness	After attention	None built-in
Purpose	Feature transformation	Prediction

Conceptual Difference

Transformer FFNN:

- Refines each token's contextual meaning
- Does not mix tokens
- Operates after attention

MLP Classifier:

- Takes all features at once
- Outputs a class or value
- Used at the end of a model

Numerical Example: Setup

Assume attention has already produced a token vector:

$$x = \begin{bmatrix} 1 & 2 \end{bmatrix}$$

Let the FFNN be:

$$\text{FFNN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2$$

Step 1: First Linear Layer

Let:

$$W_1 = \begin{bmatrix} 1 & 0 & -1 \\ 0 & 1 & 1 \end{bmatrix}, \quad b_1 = \begin{bmatrix} 0 & 0 & 0 \end{bmatrix}$$

Compute:

$$xW_1 = [1\ 2] \begin{bmatrix} 1 & 0 & -1 \\ 0 & 1 & 1 \end{bmatrix} = [1,\ 2,\ 1]$$

Step 2: Apply Activation (ReLU)

$$z = \text{ReLU}([1,\ 2,\ 1]) = [1,\ 2,\ 1]$$

All values are positive, so ReLU leaves them unchanged.

Step 3: Second Linear Layer

Let:

$$W_2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & -1 \end{bmatrix}, \quad b_2 = \begin{bmatrix} 0 & 0 \end{bmatrix}$$

Compute:

$$zW_2 = [1\ 2\ 1] \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & -1 \end{bmatrix} = [2,\ 1]$$

Final Output of FFNN

$$\text{FFNN}(x) = [2,\ 1]$$

Interpretation

- Attention gathered contextual information
- FFNN transformed it nonlinearly
- Output is a refined token representation

Analogy

- **Attention**: deciding what to focus on
- **FFNN**: thinking deeply about it

Both are needed for understanding and reasoning.

Scaled Dot-Product Attention

For queries Q, keys K, values V:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

Interpretation:

- $QK^\top$: similarity between tokens
- Softmax: attention weights
- V: information aggregation

Causal (Masked) Attention

GPT uses a **causal mask**:

$$A_{ij} = \begin{cases} -\infty, & j > i \\ 0, & j \leq i \end{cases}$$

$$\text{softmax}\left(\frac{QK^\top + A}{\sqrt{d_k}}\right)$$

Ensures:

- No access to future tokens
- Proper autoregressive generation

Why Multi-Head Attention?

Each head:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Concatenation:

$$\text{MHA}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O$$

Benefit:

- Different heads learn different relations
- Syntax, semantics, position, logic

In One-Sentence

In a Transformer, the FFNN is a position-wise MLP that applies nonlinear transformations to each token after attention has incorporated contextual information, making the model expressive and trainable.

Notation for Gradients

Let $\mathcal{L}$ be the scalar loss.
We will derive:

$$\frac{\partial \mathcal{L}}{\partial O}, \frac{\partial \mathcal{L}}{\partial A}, \frac{\partial \mathcal{L}}{\partial S},$$
$$\frac{\partial \mathcal{L}}{\partial Q}, \frac{\partial \mathcal{L}}{\partial K}, \frac{\partial \mathcal{L}}{\partial V}.$$

We assume we are given $\frac{\partial \mathcal{L}}{\partial O}$ from higher layers (backprop).

Step 1: Gradient w.r.t. V

Recall:

$$O = AV.$$

Think row-wise: $O_i = \sum_j A_{ij} V_j$.
Matrix differential:

$$dO = dA\, V + A\, dV.$$

Using $\mathcal{L}$ and inner product identity:

$$\frac{\partial \mathcal{L}}{\partial V} = A^\top \frac{\partial \mathcal{L}}{\partial O}.$$

Thus:

$$\boxed{\frac{\partial \mathcal{L}}{\partial V} = A^\top \frac{\partial \mathcal{L}}{\partial O}}$$

Step 2: Gradient w.r.t. A

From $O = AV$ again:

$$\frac{\partial \mathcal{L}}{\partial A} = \frac{\partial \mathcal{L}}{\partial O} V^\top.$$

In matrix form:

$$\boxed{\frac{\partial \mathcal{L}}{\partial A} = \frac{\partial \mathcal{L}}{\partial O} V^\top}$$

Now we need to backpropagate through the softmax:

$$A = \text{softmax}(S).$$

Step 3: Gradient w.r.t. S (Per Row)

We have:

$$A_i = \text{softmax}(S_i) \quad \text{for each row } i.$$

Let $g_i = \frac{\partial \mathcal{L}}{\partial A_i}$ be the gradient for row i.
Using the softmax Jacobian:

$$\frac{\partial \mathcal{L}}{\partial S_i} = \left(\frac{\partial A_i}{\partial S_i}\right)^\top g_i = (\text{diag}(A_i) - A_i A_i^\top)\, g_i.$$

So row-wise:

$$\boxed{\frac{\partial \mathcal{L}}{\partial S_i} = (\text{diag}(A_i) - A_i A_i^\top)\, \frac{\partial \mathcal{L}}{\partial A_i}}$$

This is applied for each row $i = 1, \ldots, n$.

Step 4: Scores from Q and K

Scores:

$$S = \frac{QK^\top}{\sqrt{d_k}}.$$

Element-wise:

$$S_{ij} = \frac{1}{\sqrt{d_k}} \langle Q_i, K_j \rangle.$$

Differential:

$$dS = \frac{1}{\sqrt{d_k}} (dQ\, K^\top + Q\, dK^\top).$$

We now find $\frac{\partial \mathcal{L}}{\partial Q}$ and $\frac{\partial \mathcal{L}}{\partial K}$.

Step 5: Gradient w.r.t. Q

Using $dS = \frac{1}{\sqrt{d_k}}(dQ\,K^\top + Q\,dK^\top)$:

$$\frac{\partial \mathcal{L}}{\partial S} : dS = \frac{1}{\sqrt{d_k}} \left(\frac{\partial \mathcal{L}}{\partial S} : dQ\,K^\top + \frac{\partial \mathcal{L}}{\partial S} : Q\,dK^\top \right),$$

where $A : B = \text{trace}(A^\top B)$ is Frobenius inner product.
Focus on dQ term:

$$\frac{\partial \mathcal{L}}{\partial S} : dQ\,K^\top = \text{trace}\left(\left(\frac{\partial \mathcal{L}}{\partial S} \right)^\top dQK^\top \right).$$

Rearranging:

$$= \text{trace}\left(K^\top \left(\frac{\partial \mathcal{L}}{\partial S} \right)^\top dQ \right) = \left(\frac{\partial \mathcal{L}}{\partial Q} \right) : dQ.$$

Thus:

$$\boxed{\frac{\partial \mathcal{L}}{\partial Q} = \frac{1}{\sqrt{d_k}} \frac{\partial \mathcal{L}}{\partial S} K}$$

Step 6: Gradient w.r.t. K

Similarly, for the term with dK:

$$\frac{\partial \mathcal{L}}{\partial S} : Q\, dK^\top = \text{trace}\left(\left(\frac{\partial \mathcal{L}}{\partial S}\right)^\top Q\, dK^\top\right) = \text{trace}\left(Q^\top \frac{\partial \mathcal{L}}{\partial S}\, dK\right).$$

Thus:

$$\boxed{\frac{\partial \mathcal{L}}{\partial K} = \frac{1}{\sqrt{d_k}} \left(\frac{\partial \mathcal{L}}{\partial S}\right)^\top Q}$$

Now we have gradients for Q, K, and V.

Projection Weights W^Q, W^K, W^V

Recall:

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V.$$

Using standard linear layer gradient:

$$\frac{\partial \mathcal{L}}{\partial W^Q} = X^\top \frac{\partial \mathcal{L}}{\partial Q},$$

$$\frac{\partial \mathcal{L}}{\partial W^K} = X^\top \frac{\partial \mathcal{L}}{\partial K},$$

$$\frac{\partial \mathcal{L}}{\partial W^V} = X^\top \frac{\partial \mathcal{L}}{\partial V}.$$

And gradient w.r.t. input X (for backprop to previous layer):

$$\frac{\partial \mathcal{L}}{\partial X} = \frac{\partial \mathcal{L}}{\partial Q}(W^Q)^\top + \frac{\partial \mathcal{L}}{\partial K}(W^K)^\top + \frac{\partial \mathcal{L}}{\partial V}(W^V)^\top.$$

Complete Gradient Flow (Self-Attention Block)

1. Start from $\frac{\partial \mathcal{L}}{\partial O}$ from upper layers.
2. Compute
$$\frac{\partial \mathcal{L}}{\partial V} = A^\top \frac{\partial \mathcal{L}}{\partial O}, \quad \frac{\partial \mathcal{L}}{\partial A} = \frac{\partial \mathcal{L}}{\partial O} V^\top.$$
3. Use softmax Jacobian row-wise to get $\frac{\partial \mathcal{L}}{\partial S}$.
4. From $S = \frac{QK^\top}{\sqrt{d_k}}$ compute:
$$\frac{\partial \mathcal{L}}{\partial Q} = \frac{1}{\sqrt{d_k}} \frac{\partial \mathcal{L}}{\partial S} K, \quad \frac{\partial \mathcal{L}}{\partial K} = \frac{1}{\sqrt{d_k}} \left(\frac{\partial \mathcal{L}}{\partial S} \right)^\top Q.$$
5. Finally, propagate to W^Q, W^K, W^V and X.

GPT-Style Models

By the end of this lecture, students will be able to:

- Explain how GPT-style models work at a systems level
- Distinguish training a GPT from using a GPT
- Build a document-based chatbot using PyTorch
- Implement Retrieval-Augmented Generation (RAG)
- Critically evaluate limitations of LLM-based systems

What Is ChatGPT?

ChatGPT is **not** a single model.

It is a system composed of:

- A **decoder-only Transformer** (GPT)
- Trained via **next-token prediction**
- Fine-tuned for instruction following
- Often combined with **retrieval systems**

Key idea: *Modern LLM applications are systems, not just models.*

Why We Do NOT Train GPT from Scratch (in Class)

Training a real GPT requires:

- Billions of tokens
- Thousands of GPU hours
- Massive infrastructure

Educational strategy:

- Use a pretrained GPT
- Teach **how to adapt it to new knowledge**

This leads to **Retrieval-Augmented Generation (RAG)**.

Chat-GPT over a Document: System Architecture

Pipeline:

1. Input document (PDF, text)
2. Chunk the document
3. Embed each chunk
4. Store embeddings in a vector database
5. Retrieve relevant chunks for a query
6. Feed context + question to GPT
7. Generate answer

Step 1: Document Ingestion

- Convert PDF to raw text
- Clean formatting artifacts
- Preserve semantic structure

Why this matters:

- LLMs do not read PDFs
- Garbage in $\Rightarrow$ garbage out

Step 2: Chunking the Document

Why chunking is required:

- GPT has a limited context window
- Documents may exceed thousands of tokens

Chunking strategy:

$$\text{Document} \rightarrow \{\text{Chunk}_1, \ldots, \text{Chunk}_N\}$$

Often with **overlap** to preserve context.

Step 3: Text Embeddings

Each chunk is converted into a vector:

$$z_i = f_{\text{embed}}(\text{chunk}_i)$$

Properties:

- Semantic similarity $\Rightarrow$ vector similarity
- Enables fast retrieval

Step 4: Retrieval via Vector Search

Given a query q:

$$z_q = f_{\text{embed}}(q)$$

Retrieve:

$$\text{Top-}k = \arg\max_i \; z_q^\top z_i$$

This replaces "memory" with search.

Step 5: Generation with GPT

The prompt is:

Context (retrieved chunks) + Question

GPT generates:

$$P(x_{t+1} \mid x_1, \ldots, x_t)$$

Important: GPT does not know the document — it only sees the prompt.

Strengths and Limitations

Strengths:

- No retraining required
- Interpretable retrieval
- Domain adaptation

Limitations:

- Retrieval errors propagate
- Hallucinations still possible
- Context window limits

Key Takeaways

- ChatGPT-style systems combine models + retrieval
- RAG is the dominant practical architecture
- Understanding systems - memorizing APIs

Core message: *Modern AI intelligence is engineered, not magical.*

What Is Retrieval-Augmented Generation (RAG)?

Definition:

Retrieval-Augmented Generation (RAG) is a system that combines

- **Information retrieval** (searching external documents)
- **Text generation** (using a GPT-style model)

Key idea:

Generation is grounded in retrieved evidence

GPT does **not memorize documents**; it reasons over retrieved context.

Why Do We Need RAG?

Limitations of plain GPT models:

- Finite context window
- No access to private or updated documents
- Hallucination when knowledge is missing

RAG solves this by:

- Injecting relevant knowledge at inference time
- Avoiding retraining or fine-tuning
- Making answers more verifiable

High-Level RAG Architecture

RAG Pipeline:

1. User asks a question
2. Relevant documents are retrieved
3. Retrieved text is added to the prompt
4. GPT generates an answer

$$\text{Query} \rightarrow \text{Retriever} \rightarrow \text{Prompt} \rightarrow \text{Generator}$$

Important: Retrieval happens *before* generation.

Step 1: Document Ingestion

Input sources:

- PDFs
- Text files
- Web pages
- Databases

Processing steps:

- Convert to plain text
- Clean formatting noise
- Preserve semantic structure

LLMs do not read documents — they read prompts.

Step 2: Chunking

Documents are split into smaller units:

$$D \rightarrow \{c_1, c_2, \ldots, c_N\}$$

Why chunking?

- Context window limits
- Efficient retrieval
- Better semantic focus

Often used: overlapping chunks to preserve context.

Step 3: Text Embeddings

Each chunk is mapped to a vector:

$$z_i = f_{\text{embed}}(c_i)$$

Properties of embeddings:

- Semantically similar text $\Rightarrow$ nearby vectors
- Language-agnostic similarity
- Fixed-length numeric representation

Step 4: Vector Database

All embeddings are stored in a vector index:

$$\{z_1, z_2, \ldots, z_N\}$$

Purpose:

- Fast similarity search
- Scales to millions of chunks

Common tools: FAISS, Annoy, Milvus, Pinecone

Step 5: Retrieval

Given a user query q:

$$z_q = f_{\text{embed}}(q)$$

Retrieve top-k chunks:

$$\arg\max_i \ \text{sim}(z_q, z_i)$$

Similarity is often:

cosine similarity or inner product

Step 6: Prompt Construction

The prompt is constructed as:

Context: *retrieved chunks*
Question: *user query*
Instruction: *answer using only the context*

Key principle:

$$\text{Prompt} = \text{Temporary knowledge injection}$$

Step 7: Generation with GPT

GPT generates tokens autoregressively:

$$P(x_{t+1} \mid x_1, \ldots, x_t)$$

Important clarification:

- GPT does not know where information came from
- It treats retrieved text as part of the conversation

Why RAG Reduces Hallucination

Hallucination occurs when:

- Model lacks relevant knowledge
- Model is forced to guess

RAG helps because:

- Relevant facts are present in the prompt
- Model is instructed to ground answers

RAG does not eliminate hallucinations, but it reduces them.

Limitations of RAG

RAG is not magic:

- Poor chunking $\Rightarrow$ poor retrieval
- Retrieval errors propagate to generation
- Long reasoning across many chunks is hard

Core limitation:

LLMs still reason only within the prompt

When Should You Use RAG?

Use RAG when:

- Knowledge changes frequently
- Data is private or domain-specific
- Interpretability is required
- Full retraining is impractical

Examples:

- CV / document chatbots
- Policy and legal assistants
- Scientific literature search

One-Slide Summary of RAG

- RAG = Retrieval + Generation
- Retrieval supplies knowledge
- GPT supplies language and reasoning
- Together, they form a practical AI system

One-line takeaway:
RAG turns a language model into a knowledge-grounded assistant.

Summary

- We derived gradients through the self-attention block:
  - Softmax over scores S
  - Bilinear form $S = QK^\top/\sqrt{d_k}$
  - Linear projections Q, K, V
- This underlies **backpropagation in Transformers**.

Questions?

References

Fleuret, F. (2024).
The Little Book of Deep Learning.
University of Geneva.

Goodfellow, I., Bengio, Y., Courville, A. (2016).
Deep Learning.
MIT Press, Cambridge.

Bishop, C. M., & Bishop, H. (2023).
Deep Learning: Foundations and Concepts.
Springer Nature.

Prince, S. J. (2023).
Understanding Deep Learning.
MIT Press.